



FIUBA · JUNIO 2026

Expense Manager

Full-stack asistido con Claude Code

Bruno Masoller · Ingeniería de Software Asistida por IA

<https://github.com/brunomaso1/uba-mia/tree/mia-iisaia/mia-iisaia/tp-final>

Contenido de la Presentación

01

Introducción

Full-stack asistido con Claude Code

03

Funcionalidades Clave

05

Stack Tecnológico

07

Scaffolding y Convenciones de Código

09

La Carpeta .claude

Configuración y reglas

02

Objetivo y Alcance

04

Arquitectura del Sistema

06

Modelo de Datos

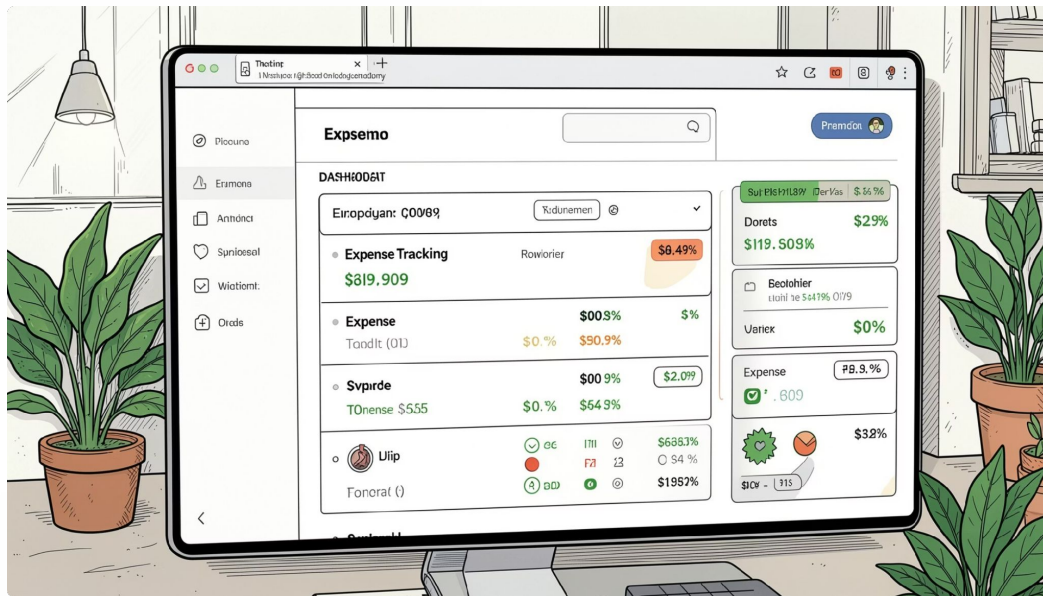
08

Desarrollo Asistido con Claude Code

10

Habilidades y Conclusiones

Objetivo y alcance



Producto

Aplicación web multiusuario para registrar y compartir gastos en grupos.

Objetivo académico

Demostrar un flujo completo de desarrollo full-stack **asistido por Claude Code**, con metodología disciplinada: especificación → plan → implementación.

Alcance funcional

Autenticación delegada, gestión de grupos y miembros, registro y consulta de gastos por grupo.

Principio de diseño

Estricta separación de responsabilidades y stack 100% containerizado con Docker Compose.



Fuera de alcance (a la fecha): liquidación/balances entre miembros, división de gastos, reportes y notificaciones.

Funcionalidades

1

Autenticación (Keycloak + PKCE)

Login delegado a Keycloak mediante el flujo PKCE; no existen formularios de login propios. Redirección automática de usuarios no autenticados.

2

Portal (/)

Página de inicio autenticada; muestra el perfil del usuario actual (GET /users/me) y permite cerrar sesión. El usuario se crea automáticamente en la base en su primer acceso.

3

Gestión de grupos

(/groups)

Crear, renombrar y eliminar grupos; listar miembros; agregar miembros desde el padrón de usuarios.

4

Gestión de gastos

(/groups/:id/expenses)

Registrar gastos (descripción, monto, fecha, grupo) y listarlos por grupo, ordenados por fecha. Acceso rápido mediante botón flotante (+).

Stack tecnológico



Frontend

Angular 22 (standalone, signals, httpResource, zoneless),
TypeScript + Angular Material



Autenticación

Keycloak 26.6.3 (OIDC / PKCE, JWT, JWKS).



Infraestructura

Docker Compose, Nginx (sirve la SPA), configuración por runtime
(config.json vía envsubst) - Kubernetes compliance.



Backend

FastAPI 0.136.3, Python 3.12+, SQLAlchemy async, Pydantic, pytest.



Persistencia

PostgreSQL 18.4, migraciones con Alembic.



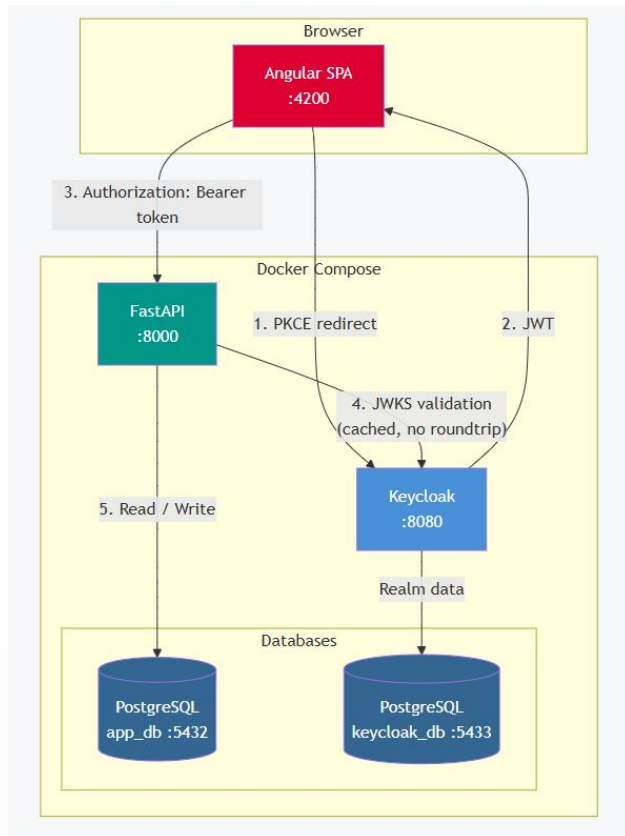
Tooling

uv (gestión de dependencias backend), Ruff (lint/format), Prettier +
ESLint, Husky (pre-commit).

Diagrama de arquitectura

Arquitectura con Docker Compose

SPA Angular, Keycloak OIDC, FastAPI y dos Postgres distribuidos.



Compose.yaml

```
1 name: expense-app
2
3 > Run All Services
4 services:
5   > Run Service
6   app_db:
7     image: postgres:${POSTGRES_VERSION:-18.4}
8     profiles: ["infra", "all"]
9     restart: unless-stopped
10    > environment: ...
11    > volumes: ...
12    > ports: ...
13    > healthcheck: ...
14
15   > Run Service
16   keycloak:
17     image: quay.io/keycloak/keycloak:${KEYCLOAK_VERSION:-26.6.3}
18     profiles: ["infra", "all"]
19     restart: unless-stopped
20    > environment: ...
21    > volumes: ...
22    > command: ["start-dev", "--import-realm"]
23    > ports: ...
24    > healthcheck: ...
25    > depends_on: ...
26
27   > Run Service
28   keycloak_db:
29     image: postgres:${POSTGRES_VERSION:-18.4}
30     profiles: ["infra", "all"]
31     restart: unless-stopped
32    > environment: ...
33    > volumes: ...
34    > ports: ...
35    > healthcheck: ...
36
37   > Run Service
38   backend:
39     build: ...
40     profiles: ["all"]
41     restart: unless-stopped
42    > environment: ...
43    > ports: ...
44    > depends_on: ...
45    > healthcheck: ...
46
47   > Run Service
48   frontend:
49     build: ...
50     profiles: ["all"]
51     restart: unless-stopped
52    > environment: ...
53    > ports: ...
54    > depends_on: ...
55    > backend: ...
56
57 volumes:
58   app_postgres_data:
59   keycloak_postgres_data:
```

Servicios compose.yaml:

- app_db → Servicio independiente de base de datos
- keycloak + keycloak_db → Servicios independientes de autenticación
- backend → FastAPI
- frontend → Angular 22

Notas:

- compose.yaml production ready
- manejo de ambientes:
 - local - híbrido
 - local - contenerizado
 - desarrollo
 - producción
- override de desarrollo con pgadmin -

```
1 name: expense-app
2
3 > Run All Services
4 services:
5   > Run Service
6   pgadmin:
7     image: dpape/pgadmin4:latest
8     profiles: ["infra"]
9     container_name: pgadmin_container
10    restart: always
11    > environment: ...
12    > ports: ...
13    > depends_on: ...
```

Scaffolding

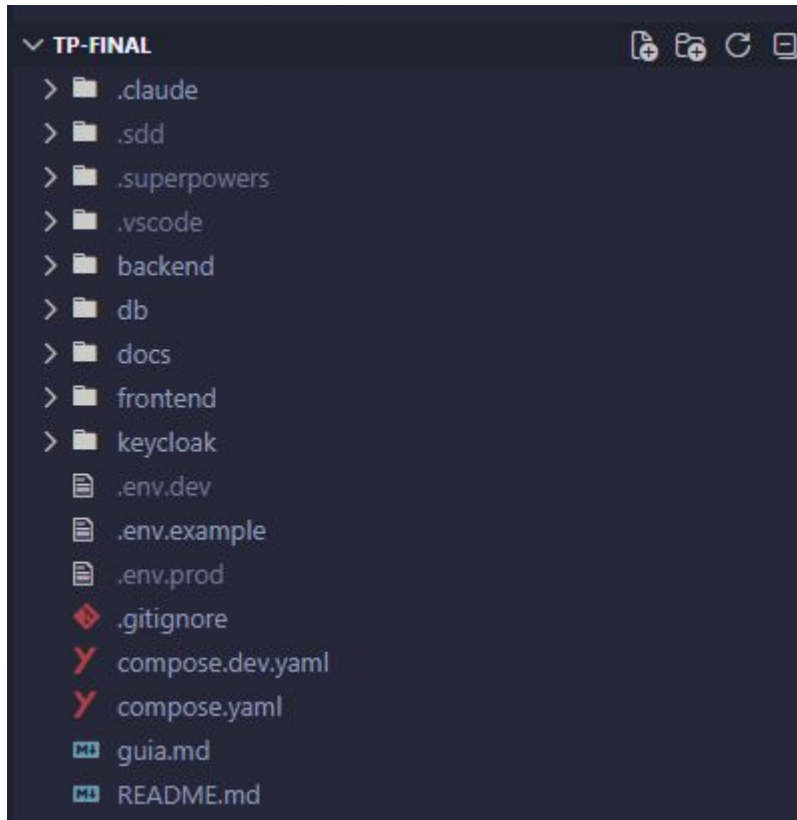
Polyrepo

 Service	 Library
 Service	 IaC
 Service	 Web
 Service	 Mobile



Monorepo

Service	Library
Service	IaC
Service	Web
Service	Mobile



Convenciones de código

Frontend — por feature

Cada feature agrupa `components/`, `pages/` y su archivo de rutas *lazy-loaded*. Artefactos de archivo único viven en la raíz del feature hasta que aparece un segundo.

Backend — por capas

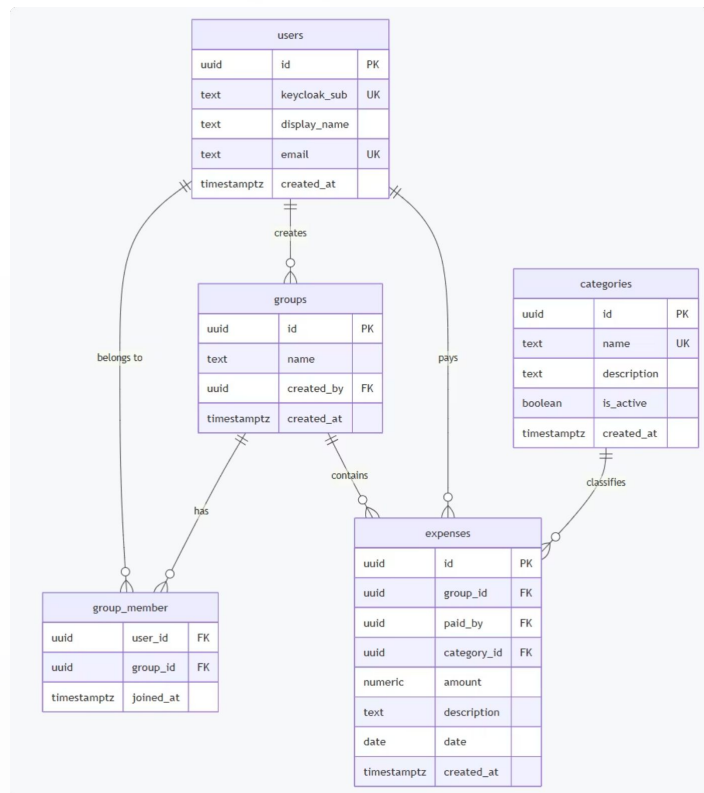
`models/` (ORM), `schemas/` (Pydantic), `routers/` (endpoints), `services/` (lógica de negocio), `deps.py` (dependencias compartidas).

- ❏ Convenciones explícitas y versionadas: reglas de estilo por área documentadas en `.claude/rules/`. Rutas REST siempre bajo `/api/v1/`; los routers devuelven schemas, no objetos ORM.

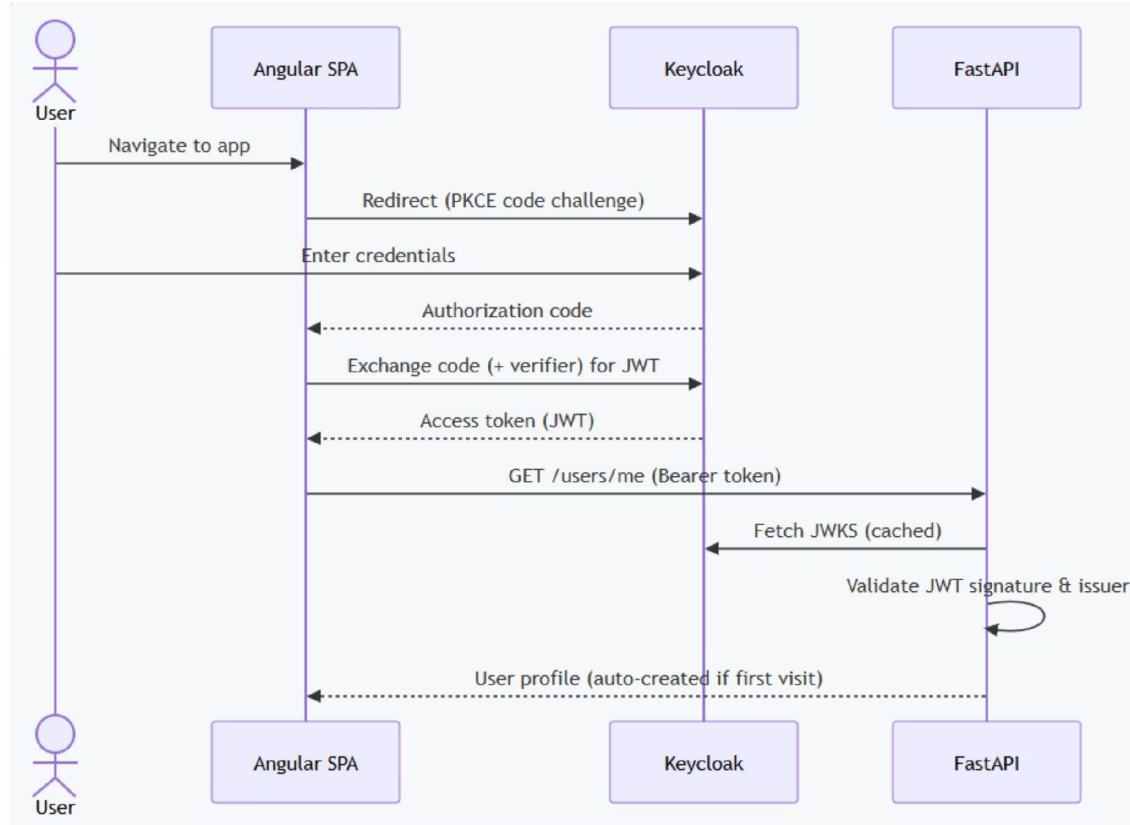
Modelo de datos

Toda evolución del esquema pasa por migraciones Alembic, nunca editando el SQL de init.

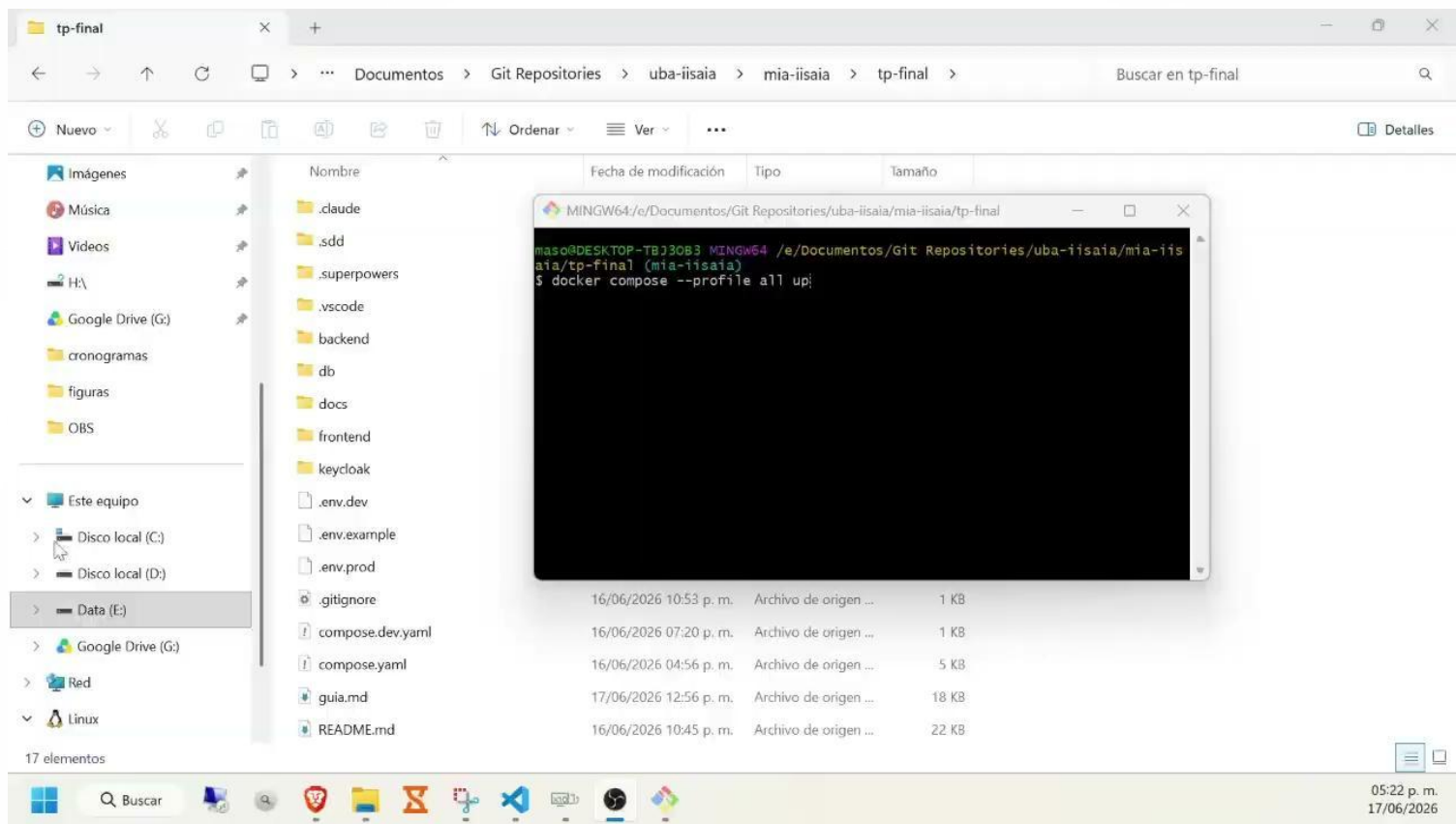
- **users** — keycloak_sub enlaza con el claim sub del JWT
- **groups** — grupos de gasto; cada uno tiene un creador
- **group_member** — relación N:N entre usuarios y grupos
- **categories** — categorías de gasto (asignación automática)
- **expenses** — gastos; pertenecen a un grupo, un pagador y una categoría



Flujo de autenticación



Demo



tp-final

Documentos > Git Repositories > uba-iisaia > mia-iisaia > tp-final >

Buscar en tp-final

Nuevo

Ordenar Ver

Imágenes

Música

Videos

H\

Google Drive (G)

cronogramas

figuras

OBS

Este equipo

Disco local (C:)

Disco local (D:)

Data (E:)

Google Drive (G)

Red

Linux

17 elementos

Nombre

Fecha de modificación

Tipo

Tamaño

.claude

.sdd

.superpowers

.vscode

backend

db

docs

frontend

keycloak

.env.dev

.env.example

.env.prod

.gitignore

compose.dev.yaml

compose.yaml

guia.md

README.md

MINGW64/e:/Documentos/Git Repositories/uba-iisaia/mia-iisaia/tp-final

```
maso@DESKTOP-TB330B3 MINGW64 /e:/Documentos/Git Repositories/uba-iisaia/mia-iisaia/tp-final (mia-iisaia)
$ docker compose --profile all up;
```

16/06/2026 10:53 p. m. Archivo de origen ... 1 KB

16/06/2026 07:20 p. m. Archivo de origen ... 1 KB

16/06/2026 04:56 p. m. Archivo de origen ... 5 KB

17/06/2026 12:56 p. m. Archivo de origen ... 18 KB

16/06/2026 10:45 p. m. Archivo de origen ... 22 KB

05:22 p. m. 17/06/2026

Made with GAMMA

- feat: add floating add-expense button to portal page brunomaso1
- feat: add 'Ver gastos' action to group card brunomaso1
- feat: add GroupExpensesPage and /groups/{id}/expenses route brunomaso1
- refactor: restore httpResource in expense dialog, fix test-1 setup brunomaso1
- feat: add ExpenseFormDialogComponent for creating expenses brunomaso1
- feat: add ExpensesService for group-scoped expense API calls brunomaso1
- feat: add expenses router nested under groups brunomaso1
- fix: handle concurrent default-category creation race brunomaso1
- feat: add expense service with default-category assignment brunomaso1
- feat: add expense request/response schemas brunomaso1
- docs: add expense management implementation plan brunomaso1
- docs: add expense management design spec brunomaso1
- fix: prevent duplicate user creation race on first login brunomaso1
- fix: correct pgadmin compose profile to infra brunomaso1
- chore: 🛠 manual-fix -> Mejoras al compose con perfiles brunomaso1
- test: add behavioral assertions to group component specs brunomaso1
- fix: add membership authorization check to group endpoints brunomaso1
- docs: add Group Management feature to README brunomaso1
- feat: wire groups routes, update nav and portal page brunomaso1
- feat: add GroupDetailPage brunomaso1
- feat: add GroupsPage brunomaso1
- feat: add GroupCard component brunomaso1
- feat: add AddMemberForm component brunomaso1
- feat: add GroupsService with mutation methods and shared interfaces brunomaso1
- feat: add group router endpoints with tests brunomaso1
- feat: add group service with tests brunomaso1
- feat: add GET /users list endpoint brunomaso1
- feat: add group and user public schemas brunomaso1
- docs: add group management implementation plan brunomaso1
- docs: add group management feature design spec brunomaso1
- fix: 🛠 manual-fix -> Tool bar color brunomaso1
- fix: use Material CSS variables for theme-safe colors in app-header brunomaso1
- feat: wire AppShell as layout route and move logout to header brunomaso1
- fix: wire app-shell.scss to component styleUrls brunomaso1
- feat: add AppShell layout component to shared brunomaso1
- feat: add AppHeaderComponent to shared brunomaso1
- docs: add header layout implementation plan brunomaso1

Desarrollo asistido con Claude Code

Metodología disciplinada

Cada feature siguió el ciclo **especificación** → **plan** → **implementación**, con documentos de diseño versionados en el repositorio.

~166 commits atómicos

Mensajes convencionales: `feat:`, `fix:`, `docs:`, `chore:`, `refactor:`, `test:`

Trazabilidad completa

El historial reconstruye la evolución del proyecto: scaffolding → autenticación → portal → grupos → gastos.

Calidad automatizada

Book pre-commit (Husky) que ejecuta Ruff (backend) y lint-staged/Prettier+ESLint (frontend) en cada commit.

La carpeta .claude: configuración y reglas

CLAUDE.md

Instrucciones de proyecto que Claude Code carga siempre: arquitectura, comandos, decisiones de diseño y convenciones de Git. Es la "memoria" del proyecto.

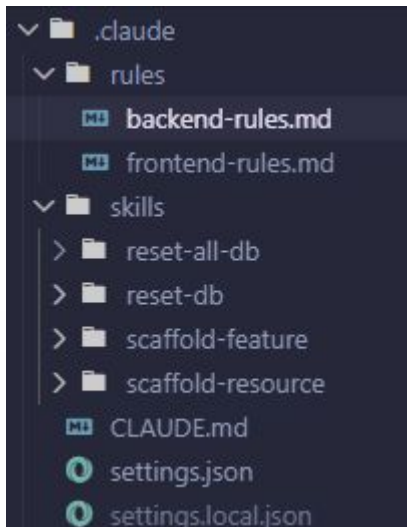
settings.json

Permisos de herramientas: lista allow (git, docker, npm, uv, pytest) y lista deny (bloquea rm -rf, git push, docker volume rm, ...).

rules/ — reglas scoped por área

frontend-rules.md (aplica a frontend/**):
Angular 22, signals, accesibilidad WCAG AA,
estructura de features.

backend-rules.md (aplica a backend/**):
FastAPI, SQLAlchemy async, Alembic, diseño
de API.



- [claude.md](#)
- [frontend-rules](#)
- [backend-rules](#)



Skills y conclusiones

Skills: automatizando tareas repetitivas

- **scaffold-feature** — genera un feature Angular 22 completo (rutas lazy, servicio, páginas y componentes co-localizados) respetando las convenciones del proyecto.
- **scaffold-resource** — genera un recurso REST de backend con todo su andamiaje (modelo, schemas, servicio, router bajo /api/v1, registro y tests pytest).
- **reset-db** — restaura la base de la app (app_db) a estado limpio y re-aplica los seeds; protege Keycloak.
- **reset-all-db** — reinicio total: destruye **todos** los volúmenes (app + Keycloak) y reconstruye desde cero.

Conclusiones - Beneficios y Trades off

- Rápido scaffolding.
- Superpower → planes y specs documentados / Te devora los tokens...
- Sub-agentes devoradores de tokens → Pueden quedar muertos. Se llega rápido al límite.
- Las rules pueden crecer rápidamente.
- Excelente tracking en base a commits atómicos.
- Autonomía alta = grandes revisiones de código.
- En 166 commits, solamente 4 commits de arreglos.
- Se pueden ejecutar los planes “manualmente” si te quedas sin tokens.
- La suscripción de USS 20 puede quedar corta.
- El uso de herramientas nuevas (ej: Angular 22) es posible con un buen contexto.
- Se pierde rápido el contexto global...
- Tiene en cuenta la seguridad, pero está bueno tener un skill o agente revisor de seguridad.
- Si la productividad se mide con respecto a cobertura de test, es excelente.